CLOUDWORX

Industrial Digital Twin Platform

TECHNICAL ASSESSMENT

Backend Integration Architecture and Build

For the role: Senior Backend Developer, Enterprise Integrations

Format	Part A, an architecture design document. Part B, a small working integration service. Part C, a 60-minute design review discussion in the next round.
Effort	Plan for 8 to 10 hours of focused work. You do not need to complete every optional item.
Submit	The design document as a single PDF, and a link to a Git repository for the service.
Deadline	Please return within 5 days of receipt.

1. Purpose

This assessment has three parts: a design document, a small build and a discussion of both. The design document carries the most weight.

We want to see how you design the integration layer that brings several enterprise tools, owned by other teams and vendors, into one workspace. You cannot change those systems. Your service has to respect their limits, their permissions and their outages, and still give the user a fast and coherent screen.

There is no single correct answer. We look for clarity of reasoning, sensible technology choices and honest awareness of trade-offs.

2. The Scenario

A global industrial manufacturer sells electrical drives and the cabinets they are installed in. Its sales engineers work across several enterprise tools today, each in its own window:

- a **product selection and sizing tool**: motor and load data in, a recommended drive variant out;
- a **product catalogue** with technical data, variants, options and 3D models;
- a **pricing and quotation system** on the company ERP;
- a **document library** of datasheets, certificates and wiring diagrams;
- the **project records** in the company's main sales application, which we call the **host application**.

The new workspace brings these together on one screen. The frontend is a 2D web application with a **3D electrical room** in the centre, where the engineer places and arranges drives and cabinets on a floor grid, and a **tool sidebar** on the right. Each sidebar panel is an integration with one enterprise tool and follows the item selected in 3D. Your job is the backend: the integration layer between that frontend and the enterprise systems.

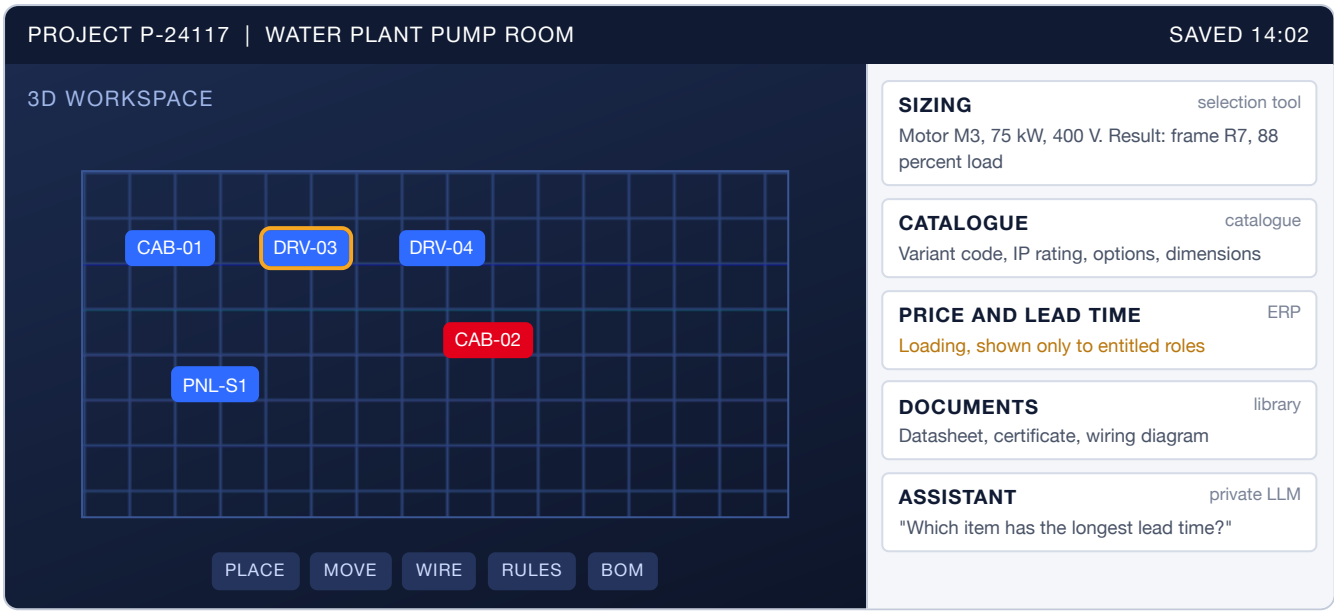


Figure 1: Layout of the workspace. Selecting DRV-03 in 3D fills every sidebar panel for that item. The red item has a rule violation.

Typical interactions:

- The engineer clicks a drive in 3D. The selection is instant and the sidebar fills with its sizing result, catalogue data, price and lead time, and documents.
- The engineer raises the motor load in the sizing panel. The drive is re-sized to a larger frame, the 3D model swaps to the new frame, and the bill of materials and price update.
- The engineer asks the assistant "What will this layout cost, and which item has the longest lead time?"

Systems and Numbers

Sizing tool API	REST, called with the user's delegated token. Limit of 20 requests per second for your whole service. A sizing run takes 1 to 6 s. One-hour maintenance window every night. Versioned, with 90 days of notice before breaking changes.
Catalogue	300 product models and 12,000 variant codes. A nightly bulk export of about 400 MB plus a change feed during the day. 3D models as glTF, 2 to 40 MB each, with three levels of detail.
Pricing (ERP)	Prices by country and customer group, with lead times. Limit of 5 requests per second. A quoted price is valid for 24 hours. Channel partners and some roles must never see prices.
Document library	80,000 files with per-document permissions. Downloads through short-lived signed URLs.
Host projects API	Source of truth for projects and their saved layouts. Sends webhooks when a project changes elsewhere.
Identity	The customer's identity provider speaks OpenID Connect. Users carry a role (sales engineer, channel partner, administrator) and a country.
Users	400 sales engineers in the pilot, 5,000 within a year, across 30 countries, with up to 600 concurrent sessions at peak.

Experience target	After a selection in 3D, the sidebar shows useful content within 1 s at p95. A panel with partial data is better than a blank panel or a spinner on every panel.
AI and hosting	A private LLM endpoint inside the customer's own cloud tenant; no customer data may leave it. Containers in the customer's Azure subscription behind their web application firewall. A security review and a penetration test gate every release.

3. What Is Fixed

FIXED

- The enterprise systems stay as they are. You consume their APIs; you cannot change them.
- The host owns projects; the sizing tool owns sizing results.
- OpenID Connect with the customer's identity provider.
- Containers on the customer's Azure tenant, with the LLM inside it.
- A React frontend with the 3D workspace and the tool sidebar.

YOUR CHOICE

Language, framework, gateway or backend-for-frontend pattern, data stores, caching, messaging, the item and BOM data model, the adapter design, the agent design and everything else.

Our own stack is Node.js, Express, MongoDB and Redis. A well-argued case for a different stack is more valuable to us than a rehearsed match to it.

4. Part A: Architecture Design Document

Your document should clearly cover each of the following.

1. **Integration architecture.** One backend-for-frontend, a gateway with an adapter per system, or something else, and why. The contract between a sidebar panel and the backend. How a sixth tool, for example a harmonics calculator, is added next to the others without changing them.
2. **Identity and entitlements.** How tokens flow from the host into your backend and onward to each system: token exchange, delegated tokens or service accounts, and where each fits. How price visibility by role and country and per-document permissions are enforced in the backend rather than only hidden in the UI.
3. **Linking the 3D scene to enterprise data.** How one placed item maps to a catalogue variant, a sizing result, a BOM line, a price and its documents. What happens when a re-size changes the variant, and how the layout, the BOM and the price stay consistent.
4. **Latency and the sidebar.** Fan-out on selection, what you cache for each system and for how long, shared versus per-user caches, partial responses with per-panel loading and failure states, and how slow sizing runs and price changes reach the screen.
5. **Limits, outages and versions.** Staying within 20 and 5 requests per second with 600 users, queuing and priority, the nightly maintenance window, and how you catch a vendor API change before your users do.
6. **Catalogue sync.** Bulk export plus change feed into your own read model, search across 12,000 variants, and mapping each variant to its 3D model and levels of detail.
7. **AI assistant.** Tools (MCP or an equivalent tool-calling design) built on the same adapters, so the assistant can never see a price or document the user cannot see. How write actions such as a re-size are confirmed by the user, how you defend against prompt injection through catalogue text or document content, and what you audit.
8. **Security, operations, failure and scale.** Secrets, logging, observability, CI/CD, what the penetration test is likely to find, behaviour when each system fails, and what breaks first at 5,000 users and at 50,000.

DELIVERABLE A1

Architecture Diagram

One diagram showing the frontend, your services, each enterprise system and the identity provider, with data flow, protocols and trust boundaries on every connection. Any tool is fine. Legibility matters more than polish.

DELIVERABLE A2

Written Explanation

- The technology choice for each component and the reasoning behind it.
- Sequence 1: the engineer clicks a drive in 3D and the sidebar fills, including the case where the sizing tool is down.
- Sequence 2: the engineer raises the motor load, the drive is re-sized to a larger frame, the 3D model swaps, and the BOM and price update.
- Sequence 3: the assistant answers "What will this layout cost, and which item has the longest lead time?" for a sales engineer who can see prices, and for a channel partner who cannot.
- Trade-offs you considered and explicitly rejected, and your assumptions, stated plainly.

5. Part B: Working Integration Service

Build a small integration service that feeds the sidebar. Budget about 4 hours. Language and framework are your choice; Node.js with TypeScript is our default. We will run your code.

REQUIRED Core

1. Mock three systems yourself: a sizing tool that is slow (0.5 to 3 s) and fails 20 percent of requests, a catalogue backed by a static JSON file, and a pricing service with a rate limit that returns prices by country.
2. A sidebar endpoint for one placed item that aggregates catalogue, sizing and pricing within a 1 s budget, with a status per section (ok, stale, unavailable, not permitted) so the frontend can render partial results.
3. A re-size endpoint: it calls the sizing mock and, if the recommended variant changes, updates the item's variant and 3D model reference and returns the BOM change.
4. A project BOM endpoint with prices filtered by the caller's entitlement, read from a role claim in a JWT. A locally signed mock token is fine.
5. An explicit cache TTL per system, retries with backoff, and a circuit breaker on the sizing tool.
6. Tests for partial results when a system fails, for entitlement filtering and for the re-size flow, plus a README that explains how to run everything.

OPTIONAL Pick Any

- An MCP server (or equivalent tool definitions) with `get_item_details` and `get_bom_cost` that honour the caller's entitlements, plus a `resize_item` write tool that will not act without an explicit user confirmation.
- Server push (SSE or WebSocket) that delivers a slow sizing result to the panel when it completes.
- An adapter registry in which adding a fourth system takes one new file and no change to the others.
- Token checks against a mock OpenID Connect issuer: signature, issuer, audience and expiry.
- A Dockerfile and a CI workflow.

A small, correct and well-tested core is worth more than a wide and fragile one.

6. Part C: Design Review Discussion

If your submission moves forward, we schedule a 60-minute session with our engineering team:

20 minutes	You walk us through your design document.
20 minutes	We change one or two constraints live, for example the pricing limit drops to 1 request per second, a harmonics calculator must join the sidebar next week, or the sizing tool changes its response format, and we adapt the design together.
20 minutes	A walkthrough of your Part B code, including one small change we ask you to make on screen.

Please have your diagram open in an editable form.

7. How We Evaluate

We are not checking whether you chose the same technologies we use.

Criterion	What we look for
Integration design	Clean boundaries per system, a clear panel contract, and a sixth tool that is easy to add.
Identity and entitlements	A correct token flow across every boundary, with permissions enforced on the server.
Data linking	One consistent identity for an item across the scene, sizing, catalogue, BOM and price, including after a re-size.
Latency and degradation	A sidebar that stays fast and useful when systems are slow, rate-limited or down.
Agent design and safety	Tools that reuse the adapters and their permissions, confirmed writes and a real answer to prompt injection.
Failure awareness	Failure modes and mitigations named before we ask. Edge cases acknowledged, not hidden.
Code quality (Part B)	Readable, tested code that runs on the first try from the README.
Clarity and taste	A design a reader can follow without questions, with fewer moving parts preferred over more.

Weighting: Part A 50 percent, Part B 30 percent, Part C 20 percent.

8. Assumptions and Submission

If something is ambiguous: state your assumption in your write-up and proceed. Making assumptions explicit is itself a signal we value. Please do not email us for clarification on the scenario.

Use of AI tools: we encourage you to use AI coding assistants and agent workflows. Say in your README which tools you used and how. In the design review we may also ask you to walk us through your AI workflow and the kind of prompts you gave it, and you should be able to explain and change any part of your submission.

How to Submit

Send the design document as a single PDF and the repository link in one email to:

- yuvraj@cloudworx.ai
- dhaivat@cloudworx.ai

Subject line: **Backend Assessment: [Your Name]**

We acknowledge every submission within one working day and respond with next steps within three working days of receipt.

Confidentiality: please treat this scenario as confidential to the hiring process. If your repository is public, keep this document out of it.